

A. I'd say the open source projects I mentioned earlier, the open source foundations we're collaborating with, and even our own graph RAG (retrieval-augmented generation) gathering—a collective of different companies and individuals driving innovation and research in graph technologies—are all helping to shape the future.

Together, they're paving the way for building an AI/ML platform and framework that will provide a stronger foundation for companies to base their enterprise LLMs on, in the future.

Q. What kind of adoption trends are you seeing when it comes to India versus the globe?

A. In terms of adoption, India closely tracks the US, Europe, and other markets, as a major portion of India's technology ecosystem is linked to global organisations, with developers collaborating across the world.

Speaking at Open Source India 2024 has led me to meet many old colleagues and friends from San Francisco, the hub where companies like LangChain, Olama, OpenAI, and others driving innovation first emerged. India feels like a very similar tech ecosystem because the same people and organisations are now operating globally.

Q. How does GraphRAG work?

A. The easiest way to think about GraphRAG is as a RAG (retrieval-augmented generation) architecture. When a question comes in from an LLM, you would want to retrieve information from the base model and access information from your internal enterprise systems—data that is not publicly available or included in the base model's training.

This process involves taking your documents—structured or unstructured, such as JSON files, PDFs, or tables—and feeding them into a knowledge graph generator. This generator creates a knowledge graph and also generates vector embeddings.

When a user submits a query, the system queries the graph database, retrieving additional context. It extracts a relevant portion of the knowledge graph, which is then passed as context to the LLM. The LLM uses this additional context to provide a more informed answer.

Q. Why not use purely generative models? Why use a combination of knowledge graphs and LLMs?

A. A purely generative AI solution excels at generating answers but often lacks specificity and accuracy. It may suffice for internal systems, such as tools to assist customer support representatives, but it is entirely unacceptable for customer-facing systems. By augmenting LLMs with a knowledge graph, you can raise the bar significantly—achieving much higher accuracy, along with explainable answers, showing exactly which portions of the knowledge graph were used to generate the results. In cases where the system fails to retrieve factual or relevant information, the query can be redirected to a human operator to ensure the question is properly addressed.

Q. How does a knowledge graph remain up to date with the latest information?

A. Knowledge graphs can be easily updated. You can run batch processes to add additional documents regularly or run queries to dynamically update the knowledge graph. There are many ways to quickly and efficiently keep the knowledge graph up to date with the latest information.

Knowledge graphs have been around for a long time, so it is well-established how to maintain them, build highly available graphs, and perform high-performance queries against them. This makes them a great complement to expert systems built on LLMs, as these systems require

up-to-date information. In enterprise use cases, it's essential that the knowledge graph is highly available—always operational and accessible to customers.

Q. What challenges do you encounter when integrating a knowledge graph with an LLM and how are you resolving them?

A. I think a knowledge graph does a reasonable job of creating nodes and relationships, and you can define the labels or schemas you want to use to generate it. This makes it a great starting point for organisations just beginning with this technology. Once people start building systems that combine knowledge graphs with LLMs, there are several approaches for integrating the two, depending on the use case. For example, you can use Text-to-Cypher, which translates the initial query into Cypher (the query language for knowledge graphs) to retrieve results from the knowledge graph.

Another approach that many organisations have found successful is using a vector database alongside the knowledge graph. Neo4j supports this, and you can link vector embeddings to the knowledge graph. When querying the vector database, relevant nodes and their related nodes are pulled back, enhancing the dataset passed to the LLM.

Some organisations use the knowledge graph as a prioritisation engine to improve the quality of the results and re-rank the results returned from the vector embedding side.

Q. What kind of industries or what kind of customers need this solution?

A. There are many different industries that can use or are already using this approach. For example, we have organisations in customer service applications using it very effectively. We've also seen it applied in research analysis—one of our customers uses